

返回 libc 攻击和 ROP

版权所有 © 2017 杜文亮,保留所有权利。

允许个人使用。如需在课堂上使用这些习题,仅当该课程采用作者的著作作为教材时方可允许。所有其他用途均须事先征得作者的同意。

S5.1. 使用 “-z noexecstack”选项编译C程序后,理论上应该会失败一个缓冲区溢出攻击(该攻击会导致易受攻击的程序返回到栈上的代码),但有些学生发现攻击仍然成功。这是什么原因呢?学生们的操作步骤完全正确。

S5.2. 在函数尾部,存储在返回地址下方区域的前一帧指针将被检索并赋值给 ebp 寄存器。然而,当返回地址溢出时,前一帧指针区域已被修改,因此在函数尾部之后,ebp 寄存器中包含的是任意值。这会有影响吗?

S5.3. 我们希望跳转到 execve() 函数来执行 “/bin/sh”,而不是跳转到 system() 函数。请描述如何实现这一点。输入中允许包含零(假设使用 memcpy() 进行内存复制,而不是 strcpy())。

S5.4. 我们知道,`system()` 函数会调用 `/bin/sh`,而 `/bin/sh` 是指向 `/bin/bash` 的符号链接。最新版本的 bash 如果检测到有效用户 ID 和实际用户 ID 不同,就会降低权限。假设我们仍然想在“返回 libc 攻击”中使用 `system()` 函数,请描述如何克服这一挑战。允许输入中包含零值(假设使用 `memcpy()` 进行内存复制,而不是 `strcpy()`)。

S5.5. 发起返回 libc 攻击时,攻击者不会跳转到 system() 函数的开头,而是使程序跳转到 system() 函数序言之后的第一条指令。请描述攻击者应如何构造输入数组。

S5.6. 地址空间布局随机化能否帮助抵御返回 libc 攻击?

S5.7. Linux 中的 ASLR 是否会随机化库函数(例如 system())的地址?

S5.8. ★ 假

设我们没有可以返回到 libc 的 system() 函数,但我们知道指令序列 A1, ..., Am, B1, ..., Bn, C1, ..., Ct 可以为我们生成一个 shell。如果该序列中的所有指令都位于连续的内存中,我们可以直接跳转到序列的开头。不幸的是,情况并非如此。该序列中的指令实际上来自三个子序列 A、B 和 C,每个子序列都位于一个函数的末尾,也就是说,每个子序列的末尾总是有一条 ret 指令。每个子序列的地址如下所示:

0xAABB1180: A1, ..., Am, 右 0xAABB2290:
B1, ..., Bn, 右 0xAABB33A0: C1, ..., Ct,
右

显然,当缓冲区溢出时,我们会将 0xAABB1180 放入返回地址。
因此,当函数返回时,它将跳转到子序列 A 的开头。
请描述一下您还会在栈上放置哪些其他值,以便当子序列 A 返回时,程序能够跳转到子序列 B;当子序列 B 返回时,程序能够跳转到子序列 B。
返回时,它将跳转到子序列 C。
上述内容称为面向返回的编程 (ROP),它是一种通用的返回到libc的技术。返回到libc的技术依赖于某些函数 (例如 system())的可用性;如果这些函数不在内存中,则

这种方法行不通。利用 ROP 技术,攻击者可以精心选择机器内存中已存在的机器指令序列,从而……

当这些序列串联起来时,它们就能达到预期的目标。这些
序列被称为小工具,它们通常以返回 (ret)指令结束;
它们位于现有程序和/或共享库代码的子程中。
ret 指令是必要的,因为 ROP 技术依赖于它从一个节点跳转到另一个节点。
后续的 ROP 设备可以串联起来,使攻击者能够
执行任意操作。使用 ROP,攻击者无需调用函数即可执行任意操作。
发起进攻。

S5.9. 函数 foo() 在将输入复制到缓冲区时存在缓冲区溢出问题

它位于其栈帧内。我们希望它返回到函数序列。
调用:bar() bar() bar() xyz(3, 5) exit()。假设我们
知道他们的地址。请描述一下你将如何利用缓冲区溢出问题来……
在 foo() 返回之前构建堆栈。你应该提供一个堆栈图。
您的答案。

```
|| 请提供 | 您的 | 答案
||
|
|
|
|<-- foo() 的 ebp
```

S5.10. 函数 foo() 在将输入复制到缓冲区时存在缓冲区溢出问题。

它位于其栈帧内。您希望它返回到库函数 xyz(0)。
你不能跳过 xyz() 函数的序言部分;也不能在输入中放入任何零。
还有一个名为 setzero(addr) 的库函数,它可以设置 4 字节的地址。
将地址 addr 处的内存内存清零。请描述您将如何构建输入。
我们并不关心程序在 xyz(0) 返回后是否会崩溃。你应该
请在答案中提供堆栈图。

```
|| 请提供 | 您的 | 答案
||
|
|
|
|<-- foo() 的 ebp
```

S5.11. 该问题在问题 S5.10 的基础上增加了一个额外的要求。函数 foo() 有一个
将输入复制到位于栈帧内的缓冲区时会发生缓冲区溢出问题。

您希望程序返回到库函数序列 `xyz(0x11111111) xyz(0) xyz(0x22222222)`。您不能跳过 `xyz()` 函数的序言部分,也不能在输入中放入任何零值。库中还有一个名为 `setzero(addr)` 的函数,可以将地址 `addr` 处的 4 字节内存设置为零。请描述您将如何构造输入。我们并不关心 `xyz(1)` 返回后程序是否会崩溃。请在您的答案中提供堆栈图。